



Seguridad de los Datos en la Nube

Informe técnico: cómo y por qué está protegida la información
del piso de máquinas

Casino Atlántico · Manatí, Puerto Rico
11 de julio de 2026 · Versión 2 (ampliada)

Basado en una auditoría de 10 revisiones independientes con verificación cruzada,
las 12 medidas de refuerzo aplicadas, y la arquitectura de Supabase/PostgreSQL.

Contenido

1. Resumen ejecutivo	2
2. Arquitectura: el camino de un dato	2
3. Metodología: la auditoría de 10 dimensiones	3
4. Las seis capas de defensa, en detalle	3
5. Cifrado en reposo y la infraestructura física	6
6. Escenarios de ataque: qué pasa si...	7
7. Hallazgos de la auditoría y medidas aplicadas	7
8. ¿Por qué la nube supera a un archivo local?	8
9. Lo que la tecnología no puede hacer sola	9
10. Glosario técnico	9
11. Conclusión	10

Los números refieren a la paginación impresa al pie de cada página.

1. Resumen ejecutivo

La aplicación del piso de máquinas del Casino Atlántico Manatí almacena su información en la nube, sobre Supabase — una plataforma gestionada que ejecuta PostgreSQL, el motor de base de datos empresarial más auditado del mundo, en centros de datos de Amazon Web Services (AWS) con certificación SOC 2 Tipo II.

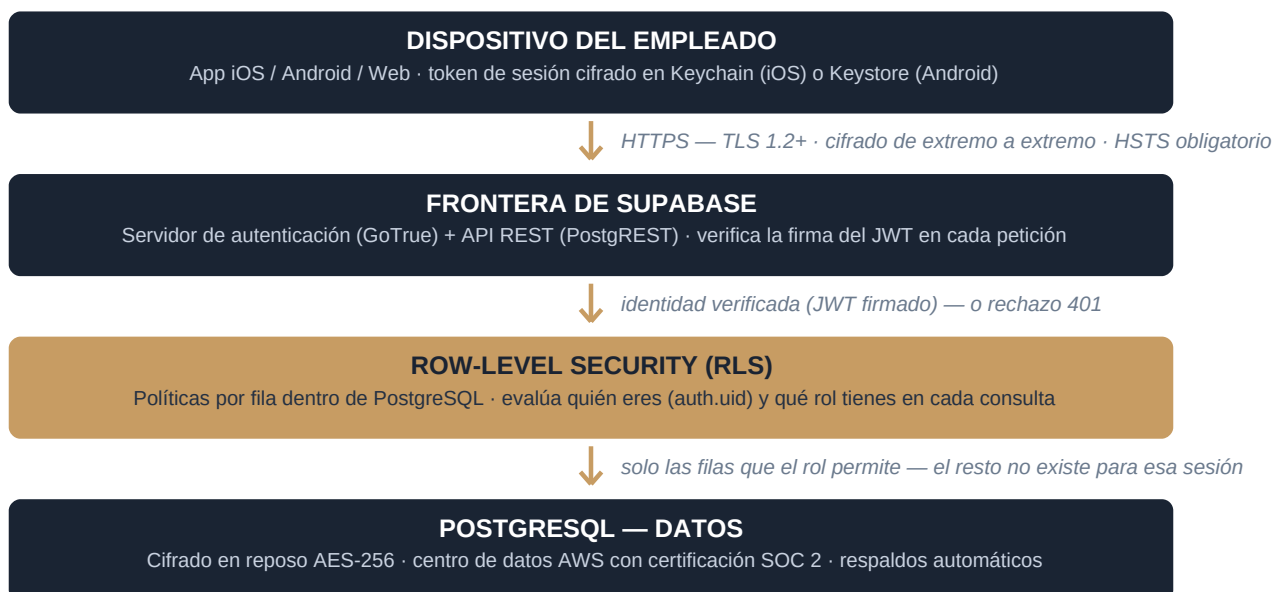
"Estar en la nube" no significa estar expuesto. Significa que los datos viven cifrados en infraestructura profesional, y que absolutamente todo acceso — venga de la app, de un navegador, de un programa hecho a mano o de un atacante — pasa por el mismo embudo: un punto de entrada único que exige identidad verificada y evalúa permisos fila por fila dentro del propio motor de la base de datos, antes de devolver un solo byte.

Para validar esta postura se ejecutó una auditoría con 10 revisiones de seguridad independientes, cada una centrada en una dimensión distinta del sistema. Un proceso de consenso cruzó los informes, resolvió desacuerdos, descartó falsas alarmas y produjo un plan de 12 medidas. Las 12 están aplicadas y verificadas: la compilación de la app, la exportación web y la verificación de tipos pasan limpias, y el despliegue quedó operativo.

Conclusión verificable: sin cuenta no se lee nada; sin rol otorgado no se lee nada; sin rol de administrador no se escribe nada; los datos viajan y descansan cifrados; y ningún secreto administrativo existe en el código, en la app instalada ni en el dispositivo.

2. Arquitectura: el camino de un dato

Cada consulta de la app recorre exactamente este camino. No existen atajos: no hay conexión directa a la base de datos, no hay puertos abiertos al público, no hay API alternativa.



Información que viaja por ese camino y queda protegida:

- **Datos operativos:**
285 máquinas (ubicación, juego, fabricante, denominaciones, apuestas) y ~25,650 registros de ingresos diarios coin-in / win — la información financiera más sensible de la operación.
- **Historial de cambios:**
compras, reubicaciones y cambios de juego con fecha y período, que revelan la estrategia comercial del piso.

- **Datos personales:**
correo, nombre y rol de cada empleado con acceso. Las contraseñas jamás se almacenan: solo un hash bcrypt (ver capa 2).

3. Metodología: la auditoría de 10 dimensiones

En lugar de una sola revisión, se lanzaron diez revisiones paralelas e independientes — cada una con acceso al código fuente completo y a las migraciones de la base de datos, y con instrucciones de ser adversarial: buscar cómo romper el sistema, no cómo justificarlo. Cada hallazgo exigía evidencia concreta (archivo y línea) y un escenario de explotación realista.

Dimensión auditada	Pregunta que respondió
1. RLS / autorización	¿Puede alguien sin sesión, o con una cuenta cualquiera, leer filas? ¿Faltan políticas?
2. Secretos	¿Hay llaves o credenciales reales en el código, el historial de git o el bundle de la app?
3. Sesión en dispositivo	¿Dónde y cómo se guarda el token? ¿Es legible si roban el equipo?
4. Flujo de autenticación	¿Resiste fuerza bruta? ¿Filtra qué correos existen? ¿El logout limpia todo?
5. Rutas y exposición	¿Se puede llegar a pantallas con datos sin sesión? ¿Se pre-descarga algo antes del login?
6. Transporte y red	¿Todo viaja por HTTPS? ¿Existen cabeceras de seguridad? ¿Hay fugas a terceros?
7. Dependencias	¿Alguna librería usada tiene vulnerabilidades conocidas (CVEs) explotables?
8. CI/CD	¿El pipeline de despliegue expone secretos o permite inyectar código?
9. Inyección / XSS	¿Hay SQL concatenado, HTML sin sanear o entradas de usuario que lleguen crudas?
10. Escalada de privilegios	¿Puede un viewer convertirse en admin o escribir saltándose la interfaz?

Un undécimo proceso (consenso) comparó los diez informes: deduplicó hallazgos repetidos, ajustó severidades cuando dos auditores discrepaban, y — importante — descartó las falsas alarmas. Por ejemplo, confirmó que la llave pública "anon key" incluida en la app NO es una vulnerabilidad (está diseñada para ser pública, porque no otorga acceso por sí sola), y que el uso de dangerouslySetInnerHTML en la plantilla web es un literal estático sin interpolación, es decir, no es un vector de XSS.

4. Las seis capas de defensa, en detalle

El diseño sigue el principio de defensa en profundidad: capas independientes, cada una suficiente por sí sola para frenar una clase de ataque, de modo que el fallo de una no arrastra a las demás. Para leer un dato hay que atravesarlas todas.

Capa 1 — Cifrado en tránsito (TLS + HSTS)

Toda comunicación entre la app y Supabase usa HTTPS sobre TLS 1.2 o superior. TLS establece un canal cifrado con intercambio de llaves efímeras (forward secrecy): aunque alguien grabara el tráfico hoy y consiguiera las llaves del servidor mañana, no podría descifrar lo grabado. El certificado del servidor se valida en cada conexión, lo que impide que un impostor se haga pasar por Supabase.

Refuerzos aplicados en esta auditoría:

- El cliente se niega a arrancar si la URL configurada no empieza con https:// — un error de configuración nunca puede degradar la conexión a texto plano.

- La web se sirve con Strict-Transport-Security (HSTS) con vigencia de 2 años, includeSubDomains y preload: el navegador recuerda que este dominio SOLO se visita cifrado y rechaza cualquier intento de conexión http:// antes de que salga a la red.

Amenaza que neutraliza: Espionaje o manipulación del tráfico en la red (ataque man-in-the-middle), incluso en WiFi público o redes internas comprometidas.

Capa 2 — Autenticación verificada (GoTrue + bcrypt + JWT)

El acceso exige correo y contraseña validados por el servidor de identidad de Supabase (GoTrue). Tres mecanismos importan aquí:

- **Hash bcrypt:**
la contraseña nunca se guarda. Se guarda el resultado de bcrypt — una función diseñada para ser computacionalmente costosa e irreversible, con "salt" único por usuario. Ni Supabase ni un atacante que robara la tabla completa podrían recuperar las contraseñas originales; probar miles de millones de combinaciones es económicamente inviable por diseño.
- **Tokens JWT firmados:**
al iniciar sesión se emite un JSON Web Token con firma criptográfica (HMAC) que caduca en aproximadamente una hora. Cada petición a la API presenta ese token; el servidor verifica la firma matemáticamente — un token alterado o inventado se detecta al instante. Del token sale auth.uid(), la identidad que las políticas RLS usan (capa 4).
- **Refresh token de un solo uso:**
la renovación automática usa un segundo token de larga vida. Con la rotación activada (tarea de panel recomendada), cada refresh token sirve UNA sola vez: si un atacante robara uno y lo usara, el sistema detecta la reutilización y revoca la familia completa de tokens.

Amenaza que neutraliza: Robo de contraseñas desde la base de datos, tokens falsificados, y reutilización de sesiones robadas.

Capa 3 — La sesión descansa cifrada en el dispositivo

Un token válido es tan sensible como una llave física. Antes de esta auditoría se guardaba en AsyncStorage — un archivo sin cifrar dentro del sandbox de la app. Ahora:

- **En iOS:**
el token vive en el Keychain del sistema, cifrado con llaves ancladas al hardware del dispositivo (Secure Enclave). No es legible desde otras apps, ni extraíble de un respaldo sin las credenciales del dueño.
- **En Android:**
se usa el Keystore del sistema, respaldado por el entorno de ejecución confiable del chip (TEE / StrongBox en equipos que lo traen). Mismo efecto: material criptográfico que no sale del hardware.
- **Detalle de implementación:**
estos almacenes limitan cada valor a 2,048 bytes y una sesión de Supabase los excede, así que el adaptador desarrollado parte el valor en fragmentos numerados y los reensambla al leer — transparente para la app, verificado por tipos.
- **En web:**
el navegador no ofrece un almacén equivalente accesible a JavaScript, por lo que ahí la protección la cargan la capa 5 (CSP que corta la exfiltración), los tokens de vida corta y la rotación de refresh tokens.

Además, cerrar sesión limpia el estado local SIEMPRE — el código lo hace en un bloque finally con alcance local, de modo que ni un fallo de red deja la sesión del usuario anterior viva en un equipo compartido del piso. Y el almacén de datos en memoria se purga en cuanto la sesión desaparece.

Amenaza que neutraliza: Robo o pérdida del dispositivo, extracción de respaldos, y sesiones que quedan abiertas en equipos compartidos.

Capa 4 — Autorización fila por fila (Row-Level Security)

Esta es la capa decisiva, y merece detalle. PostgreSQL permite adjuntar políticas de seguridad a cada tabla que se evalúan DENTRO del motor, en cada consulta, sin excepción. No es un filtro de la app que un atacante pueda saltarse: aunque alguien construya peticiones a mano contra la API con la anon key extraída del bundle, la política se ejecuta igual. El modelo es default-deny: sin una política que conceda acceso explícitamente, el resultado es cero filas.

Así luce la política real que protege la tabla de máquinas después del refuerzo (migración 0008):

```
create policy "auth_read_machines"
on public.machines for select
to authenticated
using (
  exists (select 1 from public.profiles
          where id = auth.uid()
          and role in ('admin', 'viewer'))
);
```

Lectura de la política en lenguaje llano: "concede SELECT únicamente a peticiones con sesión válida (to authenticated) cuya identidad (auth.uid, extraída del JWT firmado — no de nada que el cliente declare) tenga un perfil con rol admin o viewer otorgado". Consecuencias:

- Un visitante anónimo no cumple "to authenticated": cero filas, siempre.
- Una cuenta recién creada sin rol otorgado no pasa el exists: cero filas hasta que un administrador la reconozca.
- Las escrituras exigen rol admin con USING y WITH CHECK explícitos: un viewer que envíe un UPDATE directo a la API recibe cero filas afectadas — y la app ahora verifica ese conteo y revierte la edición en pantalla, para que una escritura negada jamás aparente haberse guardado.
- Nadie se asciende a sí mismo: la política de perfiles fija el rol con WITH CHECK — cualquier intento de cambiar el propio rol es rechazado por el motor. Otorgar o cambiar roles requiere la llave de servicio, que solo existe en el entorno administrativo protegido (capa 6).

Amenaza que neutraliza: Acceso directo a la API saltándose la app, cuentas no autorizadas, viewers que intentan escribir, y auto-promoción a administrador.

Capa 5 — El navegador blindado (cabeceras de seguridad)

La versión web se sirve desde Vercel con un conjunto estricto de cabeceras HTTP que instruyen al navegador qué permitir y qué bloquear:

Cabecera / directiva	Qué garantiza
Content-Security-Policy: connect-src	El navegador solo puede comunicarse con el propio dominio y *.supabase.co. Si un script malicioso lograra ejecutarse, no tendría a dónde enviar lo robado: la exfiltración se bloquea en el navegador mismo.
CSP: default-src / script-src / object-src	Solo se ejecuta código servido por nuestro propio dominio ('self'); los plugins y objetos embebidos quedan prohibidos (object-src 'none').
CSP: frame-ancestors + X-Frame-Options	Nadie puede incrustar la app dentro de otra página — elimina el clickjacking (superponer botones invisibles sobre la app para robar clics de un empleado con sesión).
Strict-Transport-Security	HTTPS obligatorio recordado por el navegador durante 2 años (ver capa 1).

Cabecera / directiva	Qué garantiza
X-Content-Type-Options: nosniff	El navegador no "adivina" tipos de archivo — impide que un archivo disfrazado se ejecute como script.
Referrer-Policy: no-referrer	Al seguir cualquier enlace externo, el navegador no revela la URL interna de la app.
Permissions-Policy	Cámara, micrófono y geolocalización quedan desactivados para el origen: ni el propio código puede pedirlos.

Complemento importante: las fuentes tipográficas se auto-hospedan (archivos WOFF2 servidos desde nuestro dominio). Antes se cargaban del CDN de Google; ahora la app en producción no hace NI UNA petición en tiempo de ejecución a terceros ajenos a Supabase — ni patrón de uso que filtrar, ni dependencia externa que comprometer.

Amenaza que neutraliza: Cross-site scripting (XSS) con exfiltración de datos o tokens, clickjacking sobre personal con sesión activa, y fuga de metadatos a terceros.

Capa 6 — Secretos y cadena de suministro

Existen dos llaves de poder muy distinto, y su separación es deliberada:

- **anon key (pública por diseño):**
va incorporada en la app. Solo identifica el proyecto y permite "tocar la puerta" de la API; TODO el poder de decisión está en RLS (capa 4). Extraerla del bundle no otorga ni una fila.
- **service-role key (secreta, omnipotente):**
salta RLS por completo. Por eso jamás existe en el código, en la app ni en el repositorio: vive únicamente como secreto cifrado de GitHub Actions — GitHub los cifra con libsodium antes de almacenarlos y solo los inyecta en el job autorizado.

El pipeline que puede tocar la base de datos con esa llave quedó reforzado así:

- permissions: contents: read — el token del pipeline solo puede leer el código; aunque el job se comprometiera, no puede reescribir el repositorio.
- environment: production — los secretos viven en un entorno de GitHub protegible con revisores obligatorios: un humano aprueba antes de que cualquier ejecución los reciba.
- Acciones fijadas a commit SHA inmutable (no a etiquetas móviles como @v4): nadie puede sustituir la herramienta por una versión troyanizada moviendo una etiqueta.
- npm ci --ignore-scripts en el job con la llave: las dependencias no pueden ejecutar scripts arbitrarios de instalación en el mismo proceso que sostiene el secreto.

En el lado de las librerías: overrides de npm fuerzan versiones parchadas de los componentes con avisos (dompurify >= 3.4.11, form-data >= 3.0.5, undici >= 6.27.0), el auditor de inyección confirmó que no existe SQL concatenado desde el cliente (todo pasa por la API parametrizada de PostgREST), y .gitignore cubre todo archivo .env* para que unas credenciales locales no puedan colarse a git ni por descuido.

Amenaza que neutraliza: Compromiso del pipeline de despliegue, dependencias troyanizadas (ataque de cadena de suministro), y fuga de llaves administrativas.

5. Cifrado en reposo y la infraestructura física

Las cinco primeras capas protegen el dato en movimiento y en uso. La última milla es dónde duerme:

- **AES-256 en reposo:**
los volúmenes de disco donde vive PostgreSQL están cifrados con AES-256 — el estándar aprobado para información clasificada. Un disco robado del centro de datos sería chatarra ilegible.

- **Centros de datos AWS:**
seguridad física de grado bancario (control biométrico, vigilancia continua, redundancia eléctrica y de red), sobre la que Supabase opera con certificación SOC 2 Tipo II — una auditoría externa anual que verifica los controles de seguridad de la organización, no solo sus promesas.
- **RespalDOS automáticos:**
copias de seguridad periódicas gestionadas por la plataforma, también cifradas: un fallo de hardware o un borrado accidental no destruye la información operativa del casino.
- **Aislamiento por proyecto:**
cada proyecto de Supabase es una instancia PostgreSQL dedicada — los datos del casino no comparten base de datos con ningún otro cliente de la plataforma.

6. Escenarios de ataque: qué pasa si...

La forma más honesta de evaluar una defensa es recorrer los ataques concretos que un adversario intentaría:

Si un atacante...	El sistema responde...	Capa
Intercepta el tráfico en el WiFi del casino o en uno público	Solo ve un flujo cifrado TLS indescifrable; HSTS impide degradar la conexión a texto plano.	1
Roba la base de datos de usuarios completa	Obtiene hashes bcrypt, no contraseñas: irreversibles y costosísimos de atacar por fuerza bruta.	2
Fabrica o altera un token de sesión	La firma HMAC no cuadra; la API lo rechaza con error 401 sin tocar la base de datos.	2
Roba un teléfono del piso con la app instalada	El token está en Keychain/Keystore cifrado por hardware; el dato no descargado no existe en el equipo; el token caduca en ~1 hora y la cuenta se revoca desde el panel.	3
Extrae la anon key del bundle y ataca la API directo con curl	RLS se evalúa dentro del motor: sin JWT válido con rol otorgado, cero filas. La llave pública no concede nada.	4
Crea una cuenta nueva (si el registro quedara abierto)	Sin rol otorgado en profiles, el exists de cada política falla: cero filas hasta reconocimiento manual. (Y el registro abierto se desactiva en el panel.)	4
Con cuenta viewer, envía un UPDATE directo a la API	La política de escritura exige rol admin: cero filas afectadas. La app verifica el conteo y revierte la edición fantasma en pantalla.	4
Intenta cambiarse el rol a admin	WITH CHECK fija el rol del perfil dentro del motor: rechazo inmediato. Solo la service-role key puede otorgar roles.	4
Logra inyectar un script en la web (XSS)	connect-src le niega todo destino de exfiltración salvo nuestros dominios; los tokens de vida corta acotan la ventana.	5
Incrusta la app en una página trampa para robar clics	frame-ancestors none + X-Frame-Options DENY: el navegador se niega a renderizarla embebida.	5
Compromete una librería o mueve una etiqueta de GitHub Actions	Versiones parchadas forzadas por overrides; acciones fijadas a SHA inmutable; --ignore-scripts en el job sensible; el token del pipeline es de solo lectura.	6
Busca llaves en el repositorio o su historial	No hay ninguna: auditoría de secretos limpia, .gitignore cubre .env*, y las llaves reales viven cifradas en GitHub Environments.	6

7. Hallazgos de la auditoría y medidas aplicadas

Las doce medidas del plan de consenso, todas implementadas y verificadas:

Riesgo detectado (consenso)	Nivel	Medida aplicada y evidencia
-----------------------------	-------	-----------------------------

Riesgo detectado (consenso)	Nivel	Medida aplicada y evidencia
Toda cuenta autenticada podía leer el dataset financiero completo	ALTO	Lecturas condicionadas a rol otorgado (admin/viewer) en RLS — migración 0008_harden_rls.sql. Se complementa desactivando el registro abierto (panel).
Token de sesión sin cifrar en el dispositivo	ALTO	Adaptador expo-secure-store: Keychain/Keystore cifrado por hardware, con fragmentación por el límite de 2 KB — src/lib/authStorage.ts.
Web sin cabeceras de seguridad	MEDIO	CSP completa + HSTS + X-Frame-Options + nosniff + Referrer-Policy + Permissions-Policy — vercel.json.
Pipeline CI con privilegios amplios y secretos sin gating	MEDIO	contents:read, entorno production, SHAs inmutables, --ignore-scripts — .github/workflows/.
Librerías con avisos de seguridad	MEDIO	Overrides npm a versiones parchadas; los avisos restantes son solo del toolchain de compilación y se cierran con el upgrade planificado de Expo.
Anti-auto-promoción solo implícita	BAJO	Política UPDATE con WITH CHECK que fija el rol — migración 0008.
Logout podía fallar dejando sesión viva	BAJO	Limpieza garantizada en finally con signOut de alcance local — src/store/useAuthStore.ts.
Caché visible tras cerrar sesión en equipo compartido	BAJO	Purga del almacén al perder la sesión; sin prefetch anónimo — app/_layout.tsx.
Escrituras negadas parecían guardarse (ediciones fantasma)	BAJO	Cada escritura verifica filas afectadas con .select() y revierte si el servidor negó — useSlotFloorStore.ts.
Credenciales podían colarse a git	BAJO	.gitignore cubre todo .env* preservando la plantilla .env.example.
Sin guardia contra URLs sin cifrar	BAJO	Aserción https:// al arrancar; la app falla ruidosamente ante una mala configuración — src/lib/supabase.ts.
Fuentes servidas por CDN de terceros	BAJO	WOFF2 auto-hospedadas; CSP endurecida; cero peticiones runtime a terceros — app/+html.tsx, public/fonts/.

8. ¿Por qué la nube supera a un archivo local?

La alternativa real a este sistema no es una bóveda: son hojas de cálculo en laptops y memorias USB. Comparación honesta:

Aspecto	Archivo local (Excel / USB)	Este sistema en la nube
Cifrado	Normalmente ninguno; se va con el equipo si lo roban.	TLS en tránsito + AES-256 en reposo + Keychain en el dispositivo.
Control de acceso	Quien tiene el archivo lo tiene todo, para siempre; cada copia es incontrolable.	Revocar la cuenta corta el acceso al instante, en todos los dispositivos; roles admin/viewer con granularidad por fila.
Rastro de acceso	Ninguno: imposible saber quién miró o copió qué.	Cada petición pasa por una API central con identidad verificada — hay un punto único donde mirar.
Respaldo	Manual, si alguien se acuerda; el USB se pierde.	Automático, cifrado y gestionado por la plataforma.
Parches de seguridad	La PC de oficina rara vez se parcha a tiempo.	Infraestructura parchada continuamente por Supabase/AWS bajo auditoría SOC 2.
Punto de fallo	Cientos de copias dispersas e invisibles.	Una fuente de verdad con seis capas de defensa delante.

9. Lo que la tecnología no puede hacer sola

La seguridad es un sistema socio-técnico: estas decisiones operativas viven en los paneles de Supabase y GitHub (no en el código) y completan el cierre de los hallazgos:

- **Aplicar la migración 0008 y desactivar el registro abierto (self-signup):**
juntos cierran el hallazgo de mayor riesgo. Importante: que la app no muestre pantalla de registro NO desactiva el endpoint de registro del API — hay que apagarlo en Authentication > Settings, o restringirlo a un dominio de correo corporativo.
- **CAPTCHA y límites de intentos:**
Supabase soporta hCaptcha/Turnstile en el login y límites por IP — la barrera anti fuerza bruta que el código no puede imponer solo.
- **Protección de contraseñas filtradas:**
activar el chequeo contra bases de contraseñas comprometidas conocidas (HaveIBeenPwned) para que nadie use una contraseña ya quemada.
- **Rotación de refresh tokens y vidas cortas:**
convierte cualquier token robado en un activo de un solo uso y ventana mínima.
- **CORS restringido:**
limitar los orígenes permitidos del API al dominio oficial de la app.
- **Entorno production protegido:**
en GitHub, exigir revisores para el entorno que sostiene las llaves administrativas, y activar el escaneo de secretos con push protection.
- **Higiene humana:**
contraseñas fuertes y únicas, cero cuentas compartidas, y revocación inmediata cuando alguien deja el equipo — el ataque más barato siempre será pedirle la contraseña a una persona.

10. Glosario técnico

Término	Qué significa
TLS / HTTPS	Protocolo que cifra la comunicación entre dos puntos y verifica la identidad del servidor. Es el candado del navegador.
HSTS	Cabecera que ordena al navegador usar siempre HTTPS con este dominio, incluso si el usuario teclea http://.
JWT	JSON Web Token: credencial de sesión con firma criptográfica verificable; transporta la identidad del usuario de forma infalsificable.
bcrypt	Función de hash para contraseñas, deliberadamente lenta e irreversible, con salt único por usuario.
RLS	Row-Level Security: políticas de acceso por fila evaluadas dentro de PostgreSQL en cada consulta. El corazón de la autorización.
WITH CHECK	Cláusula de una política RLS que valida los datos DESPUÉS de una escritura: aquí, impide que un perfil cambie su propio rol.
CSP	Content-Security-Policy: lista blanca que dicta al navegador qué código puede ejecutar y con quién puede comunicarse.
XSS / clickjacking	Injectar scripts en una página ajena / superponer trampas invisibles sobre una app legítima para robar interacciones.
Keychain / Keystore	Almacenes de secretos de iOS y Android, cifrados con llaves ancladas al hardware del dispositivo.
AES-256	Estándar de cifrado simétrico aprobado para información clasificada; protege los discos donde descansan los datos.
SOC 2 Tipo II	Auditoría externa anual que verifica en la práctica los controles de seguridad de un proveedor, no solo sus políticas escritas.

Término	Qué significa
anon / service-role key	Las dos llaves de Supabase: la pública solo identifica el proyecto (RLS decide todo); la de servicio salta RLS y por eso jamás sale del entorno administrativo.
CI/CD	Pipeline automatizado de integración y despliegue (aquí, GitHub Actions).
SHA pinning	Fijar herramientas del pipeline a un commit exacto e inmutable, en vez de a una etiqueta que un atacante podría mover.

11. Conclusión

La información del Casino Atlántico Manatí está protegida por seis capas independientes y verificadas: cifrado TLS en tránsito, autenticación con hash bcrypt y tokens firmados, sesión cifrada por hardware en el dispositivo, autorización fila por fila dentro del propio motor de la base de datos, un navegador restringido por políticas estrictas, y una cadena de suministro con secretos cifrados, mínimo privilegio y versiones inmutables. Debajo de todas, los datos descansan cifrados con AES-256 en infraestructura auditada bajo SOC 2.

El principio rector es que el servidor nunca confía en el cliente: cada petición demuestra criptográficamente quién es, y las reglas que deciden qué puede ver o tocar viven donde ningún atacante puede reescribirlas — dentro del motor de la base de datos.

Estar en la nube no es el riesgo: es parte de la defensa. La alternativa local — archivos dispersos, sin cifrar, sin revocación y sin rastro — es estrictamente peor en cada dimensión comparada.

Documento generado el 11 de julio de 2026 (versión 2, ampliada). Evidencia técnica en el repositorio: `supabase/migrations/0001–0008`, `src/lib/supabase.ts`, `src/lib/authStorage.ts`, `src/store/useAuthStore.ts`, `src/store/useSlotFloorStore.ts`, `vercel.json`, `app/+html.tsx`, `.github/workflows/`, y el informe de consenso de la auditoría de 10 dimensiones (PR #8).